06/03/2026

Rapport final SAE 5.02

Spécifications du Modèle de Données (MCD)

Groupe 4

RAHITA TOMMY, MONNE NOLANN et HOESCHLE YANNICK

Table des matières

1. Objectif du Modèle 3
2. Règles de Gestion (Business Rules) 3
3. Architecture du Modèle : Schéma en Flocon (Snowflake) 4
4. Dictionnaire de Données (MCD Synthétique) 5
5. Justification des Choix Techniques 6
6. Migration vers le Modèle Graphe (Neo4j) 6
6.1 Justification du Modèle Graphe 6
6.2 Processus de Migration (ETL) 8
7. Validation et Comparaison des Performances 9
7.1 Requêtes Métiers 9
7.1 Vérification du voisinage 11
8. Évolutivité et Maintenance du Système 14
Conclusion et Recommandations 14

1. Objectif du Modèle

Le modèle de données a pour but de structurer les statistiques de délinquance (2012-2021) pour permettre une analyse multidimensionnelle (OLAP). Il doit transformer des fichiers plats hétérogènes en un système centralisé garantissant :

- **L'intégrité des données :** Pas de doublons, typage fort.
- **L'historisation :** Capacité à accumuler les années successives.
- **L'analyse hiérarchique :** Agrégation par géographie (Département > Région) et par sémantique (Crime > Catégorie).

2. Règles de Gestion (Business Rules)

Les règles suivantes ont dicté la conception du modèle et les cardinalités retenues :

- **RG01 - Unicité du Fait :** Un "Fait" est défini de manière unique par la combinaison de trois dimensions : une Année, un Service et un Type de Crime.
- **RG02 - Géographie :** Un Service est rattaché à un et un seul Département. Un Département appartient à une et une seule Région administrative.
- **RG03 - Typologie :** Un Crime (ou Index) appartient à une et une seule Catégorie générique (ex: "Vols").
- **RG04 - Organisation :** Les services de Police et de Gendarmerie sont stockés dans une entité unique, distingués par un attribut de type (Force de l'ordre), afin de faciliter les agrégations nationales.

3. Architecture du Modèle : Schéma en Flocon (Snowflake)

Nous avons choisi un schéma en flocon : les catégories (comme la région ou crime) sont divisées en sous-tables pour créer des niveaux de détails précis (ex : Département > Région).

3.1 Entité Centrale (Table de Faits)

- **T_FAIT**

C'est sur cette table que tout notre système décisionnel se repose. Elle stocke les métriques brutes principales. Nous avons dans cette table une ligne pour un type de crime pour un service pour une année donnés. Nous avons ici le grain de la table (annee, id_service, id_crime) Cette table nous permet de rassembler les données de criminalité pour analyser les volumes par région, par catégorie ou par date en utilisant des clés étrangères tel que T_SERVICE et T_CRIME.

3.2 Dimensions Organisationnelles & Géographiques

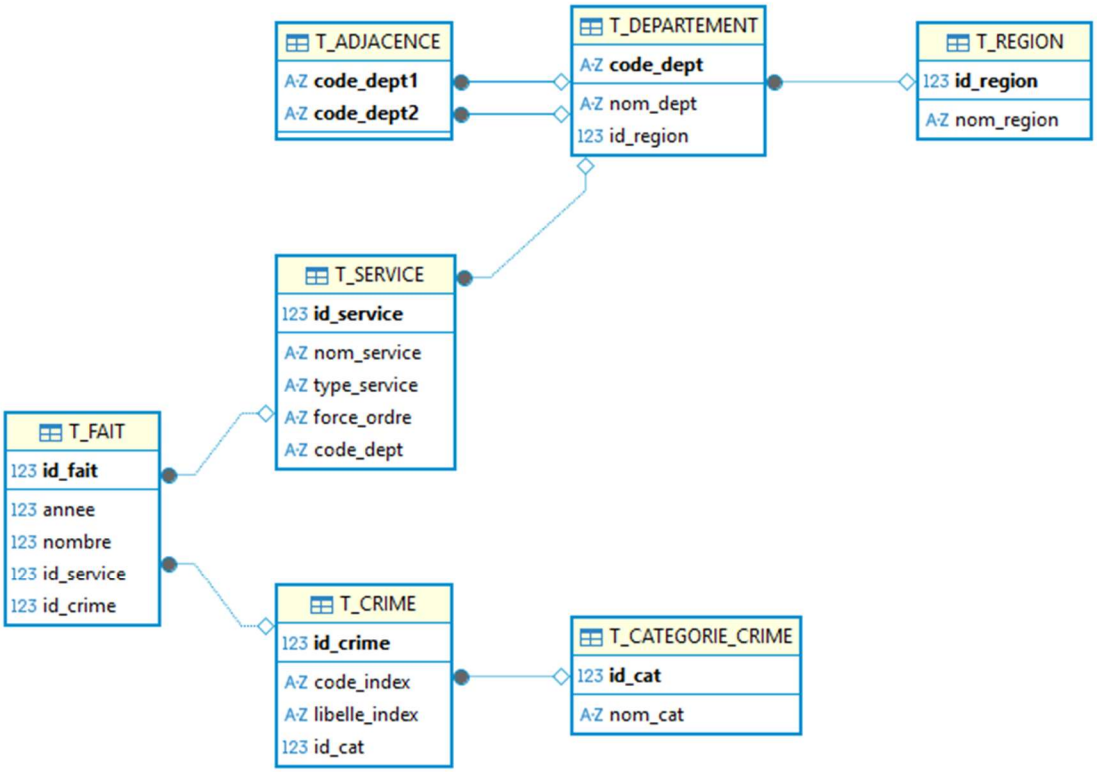
- **T_SERVICE :** Représente l'unité opérationnelle (Commissariat, Brigade, Compagnie). Attributs : Nom, Force (PN/GN), et Type (CSP, Gendarmerie).
- **T_DEPARTEMENT :** Niveau de granularité administrative. Attributs : Code INSEE (incluant la gestion des codes 2A/2B), Nom (Isère, Rhône, etc.).
- **T_REGION :** Ajout des régions pour avoir des analyses géographiques à un niveau plus macroscopique
- **T_ADJACENCE (Enrichissement) :** Table technique modélisant la proximité des départements. Elle liste les couples de départements limitrophes, préparant ainsi la relation [:TOUCHE] du futur graphe.

3.3 Dimensions Sémantiques

- **T_CRIME :** Nomenclature officielle du Ministère (107 index). Attributs : Code index, Libellé précis.
- **T_CATEGORIE_CRIME :** Regroupement thématique. Ce niveau simplifie l'analyse en réduisant les 107 index en 7 familles majeures.

La categorisation est faite de manière automatique lors de l'import à partir des libelles de crimes. Par exemple, un libelle contenant 'vol' ou 'cambriolage' est classe dans 'Vols et Cambriolages'. Cette méthode produit 7 catégories : Vols et Cambriolages, Violences Physiques, Violences Sexuelles, Escroqueries et Delits Financiers, Stupefiants, Degradations, et Autres Crimes et Delits. C'est une méthode qui permet de regrouper les crimes automatiquement.

4. Dictionnaire de Données (MCD Synthétique)



5. Justification des Choix Techniques

Notre architecture repose sur un schéma en flocon qui sépare strictement les régions et les catégories. Si ce choix densifie légèrement les jointures SQL, il offre une transition directe vers Neo4j en transformant chaque table de dimension en un label de graphe indépendant. Pour optimiser les performances, nous avons fusionné la Police et la Gendarmerie au sein d'une table unique (`T_SERVICE`) différenciée par un simple marqueur. Cette structure permet des agrégations nationales immédiates, évitant ainsi les opérations de fusion (`UNION`) coûteuses en ressources.

L'anticipation du modèle de graphe se traduit également par la création de la table `T_ADJACENCE`. En SQL, elle gère les relations de voisinage entre départements, mais son véritable potentiel se révèle lors de la migration : elle devient une relation physique `[:TOUCHE]` améliorant la vitesse des analyses spatiales. Enfin, l'intégrité de l'ensemble est verrouillée par un principe d'idempotence sur le triplet (année, service, crime). Ce mécanisme de sécurité garantit la fiabilité des statistiques ministérielles en filtrant automatiquement les doublons lors des imports de fichiers.

6. Migration vers le Modèle Graphe (Neo4j)

6.1 Justification du Modèle Graphe

Le passage d'un modèle relationnel à un modèle orienté graphe transforme des lignes de tableaux statiques en un réseau dynamique d'entités interconnectées. Cette approche ne se contente pas de stocker les données, elle modélise les relations métiers comme des citoyens de premier rang.

Transformation des entités et Schéma

Le modèle a été restructuré autour de 5 types de nœuds et 5 types de relations :

- **Nœuds :** `Region`, `Departement`, `Service`, `Crime`, `Categorie`.
- **Relations :**
  - `(Service)-[:SITUE_DANS]->(Departement)-[:APPARTIENT_A]->(Region)`
  - `(Service)-[:A_ENREGISTRE {annee: 2021, nombre: 45}]->(Crime)`
  - `(Crime)-[:TYPE_DE]->(Categorie)`
- **Enrichissement Spatial (Exclusivité Graphe) :**
  - `(Departement)-[:TOUCHE]-(Departement)` (liens en département limitrophes).

Justification technique et opérationnelle

Dans le modèle graphe, la donnée (nombre, annee) est portée directement sur la relation [:A_ENREGISTRE], ce qui simplifie le parcours Service vers Crime. Pour les requêtes d'agrégation classiques cependant, les deux modèles donnent des résultats identiques.

La relation `[:TOUCHE]` permet de répondre à la question : *"Quels sont les crimes commis dans les départements limitrophes du Rhône ?"* sans aucun calcul de coordonnées ni jointures coûteuses. Neo4j utilise

le "pointeur physique" pour sauter d'un département à ses voisins, garantissant un temps de réponse constant même sur des volumes massifs.

Ce schéma est aussi dit "flexible" (Schema-free). Si demain le Ministère souhaite ajouter des données sur les effectifs des brigades ou des relations de coopération entre services, ces propriétés peuvent être ajoutées aux nœuds `Service` sans migrer toute la structure de la base, contrairement à SQLite qui nécessiterait une modification lourde du DDL (`ALTER TABLE`).

Le langage Cypher permet d'exprimer des requêtes complexes plus simplement. Une analyse qui nécessite 4 jointures en SQL (environ 15 lignes de code) peut s'écrire en uniquement 3 lignes en Cypher, facilitant la maintenance par les analystes de la DGDSN.

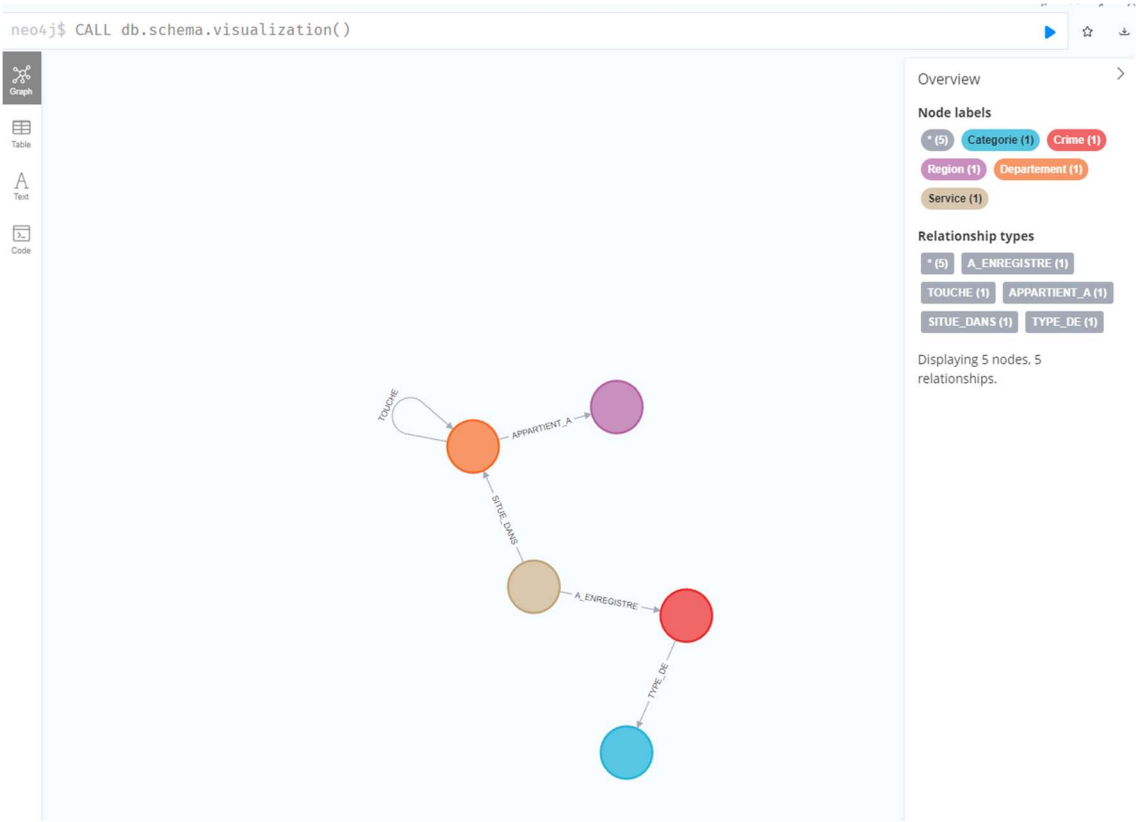
Analyse stratégique des deux modèles

Pour les analyses statistiques classiques (totaux par région, évolution annuelle, classements), les deux modèles donnent des résultats identiques. C'est ce que montrent les requêtes 1 à 10 : les chiffres sont exactement les mêmes en SQL et en Cypher. Sur ce type de requête, SQLite est même plus adapte car il est optimisé pour les agrégations sur de gros volumes de données.

Le graphe apporte une valeur ajoutée réelle uniquement pour les analyses de relations entre entités. Par exemple, la question 'quels crimes sont commis dans les départements voisins du Rhone ?' nécessite en SQL une jointure supplémentaire sur `T_ADJACENCE`. Dans Neo4j, la relation `[:TOUCHE]` permet de parcourir directement les voisins. C'est ce qu'illustrent les requêtes 11 et 12.

Nous avons conservé SQLite comme étape intermédiaire pour deux raisons : nettoyer les données brutes du fichier Excel, et disposer d'une base de référence pour vérifier que la migration n'a pas perdu de données.

Nous avons également choisi le modèle graphe pour sa capacité à représenter visuellement les relations entre entités. Sur Neo4j Browser, on peut voir directement les liens entre départements voisins et naviguer de nœud en nœud, ce qui est plus intuitif qu'un tableau de résultats SQL pour comprendre des relations spatiales.



6.2 Processus de Migration (ETL)

La migration a été réalisée via un script Python (`migration_neo4j.py`) suivant le flux suivant :

1. **Extraction :** Lecture de la base SQLite enrichie.
2. **Transformation :** Nettoyage des codes départements et catégorisation heuristique des libellés de crimes.
3. **Chargement :** Injection dans Neo4j par lots de 1000 relations pour optimiser la mémoire.

Gestion des valeurs manquantes

Le fichier Excel contient des cellules vides. Ces valeurs sont converties en chaine vide puis filtrées à trois niveaux :

- Un nom de service vide : la colonne est ignorée, aucun nœud Service n'est créé.

- Un code crime vide : la ligne est ignorée, aucun nœud Crime n'est créé.

- Une valeur numérique vide ou égale à zéro : aucune relation A_ENREGISTRE n'est créée.

Dans le graphe final, les valeurs manquantes ne sont donc pas représentées.

7. Validation et Comparaison des Performances

Les 10 premières requêtes donnent exactement les mêmes résultats en SQL et en Cypher, ce qui valide que la migration n'a pas perdu de données. L'avantage (qui reste minime) du graphe apparait uniquement sur les requêtes 11 et 12 qui exploitent le voisinage géographique : la relation `[:TOUCHE]` permet d'accéder directement aux départements voisins, sans jointure supplémentaire plus compliqué à calculer en SQL. (Toutes les requêtes sont accéssible dans le fichier : `test_requetes.md)`

7.1 Requêtes Métiers

1. Classement des départements par volume total de crimes

Objectif : Vérifier les totaux par département (toutes années confondues).

SQL

```sql
SELECT d.nom_dept, SUM(f.nombre) as total
FROM T_FAIT f
JOIN T_SERVICE s ON f.id_service = s.id_service
JOIN T_DEPARTEMENT d ON s.code_dept = d.code_dept
GROUP BY d.nom_dept
ORDER BY total DESC
LIMIT 10;
```

Résultats

```
nom_dept         |total  |
-----------------+-------+
Paris            |2624819|
Nord             |1567528|
Bouches-du-Rhône |1551583|
Seine-Saint-Denis|1400269|
Rhône            |1278887|
Hauts-de-Seine   |1064364|
Val-de-Marne     | 891165|
Gironde          | 885659|
Haute-Garonne    | 875000|
Val-d'Oise       | 817864|
```

Cypher

```
MATCH (d:Departement)<-[:SITUE_DANS]-(s:Service)-[f:A_ENREGISTRE]->()
RETURN d.nom as nom_dept, sum(f.nombre) as total
ORDER BY total DESC
LIMIT 10;
```

Résultats

nom_dept	total
"Paris"	2624819
"Nord"	1567528
"Bouches-du-Rhône"	1551583
"Seine-Saint-Denis"	1400269
"Rhône"	1278887
"Hauts-de-Seine"	1064364
"Val-de-Marne"	891165
"Gironde"	885659
"Haute-Garonne"	875000
"Val-d'Oise"	817864

7.1 Vérification du voisinage

Requête 12 - Catégories de crimes chez les voisins de l'Isère

Objectif : trouver les 5 catégories de crimes les plus fréquentes dans les départements qui touchent l'Isère (code 38).

Résultats identiques en SQL et en Cypher, seule la vitesse de calcul est différente Cypher étant légèrement plus rapide.

12. "Propagation par catégorie"

Objectif : Lister les 5 catégories de crimes les plus fréquentes dans les départements qui touchent l'Isère ('38').

SQL

```sql
SELECT cat.nom_cat, SUM(f.nombre) AS Total
FROM T_ADJACENCE adj
JOIN T_SERVICE s ON adj.code_dept2 = s.code_dept
JOIN T_FAIT f ON s.id_service = f.id_service
JOIN T_CRIME c ON f.id_crime = c.id_crime
JOIN T_CATEGORIE_CRIME cat ON c.id_cat = cat.id_cat
WHERE adj.code_dept1 = '38'
GROUP BY cat.nom_cat
ORDER BY Total DESC
LIMIT 5;
```

Résultats

nom_cat	Total
Vols et Cambriolages	1500478
Autres Crimes et Délits	480729
Dégradations	168017
Escroqueries et Délits Financiers	158865
Stupéfiants	148813

Cypher

```cypher
MATCH (d:Departement {code: '38'})-[:TOUCHE]-(voisin:Departement)
MATCH (voisin)<-[:SITUE_DANS]-(:Service)-[f:A_ENREGISTRE]->(c:Crime)-
[:TYPE_DE]->(cat:Categorie)
RETURN cat.nom AS Categorie, sum(f.nombre) AS Total
ORDER BY Total DESC
LIMIT 5;
```

Résultats

Categorie	Total
"Vols et Cambriolages"	1500478
"Autres Crimes et Délits"	480729
"Dégradations"	168017
"Escroqueries et Délits Financiers"	158865
"Stupéfiants"	148813

Requête 11 - Criminalité chez les voisins du Rhone

Objectif : trouver le nombre total de crimes dans chaque département limitrophe du Rhone (code 69).

Résultats : identiques en SQL et en Cypher. La différence est dans la façon d'écrire la requête : en SQL il faut une jointure sur la table `T_ADJACENCE`, en Cypher on parcourt directement la relation `[:TOUCHE]` entre départements, comme la requête précédente ici Cypher est légèrement plus rapide.

11. "Criminalité chez les voisins"

Objectif : Pour un département donné (ex: le Rhône '69'), trouver le nombre total de crimes enregistrés dans tous les départements limitrophes.

SQL

```sql
SELECT d2.nom_dept AS Voisin, SUM(f.nombre) AS Total_Crimes
FROM T_ADJACENCE adj
JOIN T_DEPARTEMENT d2 ON adj.code_dept2 = d2.code_dept
JOIN T_SERVICE s ON d2.code_dept = s.code_dept
JOIN T_FAIT f ON s.id_service = f.id_service
WHERE adj.code_dept1 = '69'
GROUP BY d2.nom_dept
ORDER BY Total_Crimes DESC;
```

Résultats

```
Voisin        |Total_Crimes|
--------------+------------+
Isère         |      705846|
Loire         |      356506|
Ain           |      249184|
Saône-et-Loire|      194405|
```

Cypher

```
MATCH (d1:Departement {code: '69'})-[:TOUCHE]-(voisin:Departement)
MATCH (voisin)<-[:SITUE_DANS]-(s:Service)-[f:A_ENREGISTRE]->(c:Crime)
RETURN voisin.nom AS Voisin, sum(f.nombre) AS Total_Crimes
ORDER BY Total_Crimes DESC;
```

Résultats

Voisin	Total_Crimes
"Isère"	705846
"Loire"	356506
"Ain"	249184
"Saône-et-Loire"	194405

8. Évolutivité et Maintenance du Système

La solution Graphe ne se contente pas de stocker l'existant, elle offre une flexibilité totale pour l'avenir grâce à sa structure sans schéma rigide.

8.1 Ajout de nouvelles données temporelles

Pour intégrer une nouvelle année de statistiques (ex: 2022 ou 2023), le processus est simplifié : Grâce à l'utilisation systématique de la clause `MERGE` dans nos scripts Cypher, Neo4j identifie les entités déjà existantes (Services, Départements) et ne crée que les nouvelles relations `[:A_ENREGISTRE]`. L'ajout d'une année ne nécessite aucune modification de la structure de la base (pas de `ALTER TABLE`), contrairement au modèle relationnel.

8.2 Enrichissement Géo-spatial Continu

Nous avons déjà implémenté les relations de voisinage `[:TOUCHE]`. Ce modèle peut être enrichi sans surcoût de performance pour des analyses plus fines si nécessaires on peut par exemple :

- Ajouter des relations de proximité basées sur d'autres critère (ex: `[:LIMITE_COMMUNALE]`).
- Analyser la propagation car le graphe permet d'identifier plus simplement si une hausse de la criminalité dans un département se propage aux territoires limitrophes, une opération qui demanderait des jointures SQL récursives complexes et lentes.

Conclusion et Recommandations

Ce projet nous a permis de mettre en place une chaine complète de traitement de données, depuis un fichier Excel brut jusqu'à une base de données graphe opérationnelle. La comparaison des deux modèles montre que le choix d'une technologie ne se fait pas uniquement en termes de celui qui aura toujours les meilleures performances, mais aussi en fonction du type d'analyse que l'on veut réaliser.

Pour les analyses statistiques classiques (totaux, classements, tendances annuelles), le modèle relationnel est tout aussi efficace que le graphe et souvent plus simple à interroger. Le graphe apporte un vrai avantage pour les analyses de voisinage géographique, grâce à la relation `[:TOUCHE]` entre départements, ce qui serait beaucoup plus complexe à traiter en SQL.

Le principal apport de ce projet est donc double : d'abord la mise en place d'un modèle de données propre et structure à partir de données brutes hétérogènes, ensuite l'enrichissement de ce modèle avec des relations entre départements, qui ouvre la porte à des analyses impossibles avec le seul modèle relationnel.

Voici le lien vers notre vidéo youtube : https://youtu.be/1-iSQZXKx6Q

Voici le lien vers le repo git : https://github.com/HoeschleY/SAE-Migration-de-donn-es.git